

基于空间几何分析的烟雾弹投放策略研究

摘 要

无人机投烟雾弹是一种通过无人驾驶飞行器精确部署烟幕干扰弹的防御技术，广泛应用于军事对抗、要地防护及电子战场景。本文针对无人机航向、烟幕弹投放点及起爆时序等关键参数，结合烟幕云团下沉等物理特性，对多无人机协同烟幕干扰弹投放策略优化问题展开研究，建立数学建模与智能优化算法相结合的研究框架，实现遮蔽效能最大化。

针对问题一：为解决有效遮蔽问题，建立模型对导弹与烟幕球团形成的“遮蔽视锥”进行处理，并通过证明得到：只要圆柱上下圆周均位于“视锥”内部，则整个圆柱被完全遮蔽。为精确计算，采用离散时间步进的方法，建立运动学方程与几何遮蔽条件验证模型，计算得到 FY1 投放单弹对 M1 的有效遮蔽时长为 1.3916s。

针对问题二：为解决单个无人机 FY1 运动与投弹规划，运用粒子群算法与序列二次规划的两阶段混合优化策略，优化无人机速度、方向、投放点及起爆点，实现遮蔽时间最大化。最终得到最大遮蔽时间 4.587s。

针对问题三：为解决单个无人机 FY1 投放三枚烟幕弹，引入时间协同的复杂性。由于会出现同时存在多个烟幕云团的场景，需要对问题一中的有效遮蔽模型进行修正，将圆柱体离散化，运用“射线投射法”判断是否有效遮蔽。优化算法仍然沿用问题二中的混合优化策略，最终得到最大遮蔽时间：6.45s。

针对问题四：为优化三架无人机 FY1, FY2, FY3 各投放一枚烟幕弹时的飞行策略与投放时序，以实现对单个导弹 M1 的最佳联合遮蔽。先通过问题二的模型分别计算每架无人机对导弹产生的最大遮蔽时间（此时不考虑烟幕重叠），对时间相加求得联合遮蔽的上限。对三个独立的烟幕团进行分析发现：在分别作用的策略下，烟幕团不会出现重叠情况，因此联合遮蔽上限就是最佳联合遮蔽时间：11.549s。

针对问题五：问题五进一步扩展为复杂的多对多协同干扰场景：利用全部 5 架无人机，每架至多投放 3 枚烟幕弹，同时对 M1、M2、M3 三枚来袭导弹实施干扰。需同时优化资源分配、运动参数及时序参数。面对这一超高维、混合整数非线性规划问题，采用多组并行 PSO-SQP 优化各无人机的投放策略，并简化模型，只考虑一个无人机遮蔽一种导弹，最后引入协同约束确保策略的整体可行性。最终算出最大遮蔽时间 20.40s。

关键字：遮蔽视锥 射线投射法 混合优化策略 离散时间步进

一、问题重述

1.1 问题背景

烟幕干扰装置通过化学爆燃或定向爆炸释放微粒物质，在目标区域上空快速生成高浓度烟幕或气溶胶屏障，有效阻断敌方导弹的光电、红外及雷达探测路径，兼具经济性与战术实用性^[1]。现在采用无人机进行投放操作，如何控制无人机进行投弹决策是保护、遮蔽目标的关键。

1.2 问题提出

基于上述背景，建立数学模型解决以下问题。

空域中存在半径为 7m、高度为 10m，底面圆心位于 (0,200,0) 的圆柱真目标，在 origin 处有一个为掩护真目标的假目标；共有 5 架无人机，位置信息分别为 $FY1(17800,0,1800)$ 、 $FY2(12000,1400,1400)$ 、 $FY3(6000,-3000,700)$ 、 $FY4(11000,2000,1800)$ 、 $FY5(13000,-2000,1300)$ ，速度在 70 至 140m/s 区间且飞行高度不发生改变。若需要进行连续投弹，相邻两枚烟雾的投弹间隔时间至少为 1s，干扰弹爆炸后产生以 3m/s 匀速下沉的烟幕，烟幕形成半径为 10m 的球形，能够提供 20s 的有效遮蔽；空域中还存在三枚空地导弹，位置信息分别为 $M1(20000,0,2000)$ 、 $M2(19000,600,2100)$ 、 $M3(18000,-600,1900)$ ，运动方向直指假目标，飞行速度为 300m/s。

问题一： $FY1$ 携带 1 枚烟幕干扰弹，以 120m/s 水平朝假目标飞行，在收到命令 1.5s 后投放烟幕干扰弹，3.6s 后扩张为球形烟幕，据此信息计算出干扰弹对单个导弹 $M1$ 的有效遮蔽时长。

问题二： 仍然调用 $FY1$ 携带 1 枚烟幕干扰弹对 $M1$ 进行干扰任务，优化无人机的飞行策略与单枚烟幕弹的投放时序，以最大化对单个导弹 $M1$ 的有效遮蔽时长。

问题三： 调用 $FY1$ 携带 3 枚烟幕干扰弹对 $M1$ 进行干扰任务，优化 $FY1$ 投放三枚烟幕弹时的飞行策略与各弹药的时序安排，以最大化对单个导弹 $M1$ 的协同遮蔽时长。

问题四： 调用 $FY1$ 、 $FY2$ 、 $FY3$ 三架无人机，各投放一枚烟幕弹时的飞行策略与投放时序，以实现对单个导弹 $M1$ 的最佳联合遮蔽。

问题五： 所有无人机均被调用，指导一个拥有五架无人机的无人机编队，所有无人机的烟幕干扰弹投掷上限数为 3，来应对三枚同时来袭的导弹，寻求全局最优的资源分配和部署策略。

二、问题分析

2.1 问题一的分析

首先构建运动模型，确定导弹和烟幕在不同时刻的相对位置，其可以通过建立空间坐标系，对导弹和烟幕干扰弹的运动轨迹列出函数表达式。然后建立有效遮蔽模型，将导弹和起爆前的干扰弹理想化地视为一个质点，起爆后的烟幕视为一个球体，其有效遮蔽区域为一个“遮蔽视锥”。通过建立模型判断判断圆柱是否完全位于“视锥”内部，即可判断是否遮蔽。

由于上述角度的瞬时值是随时间变化的复杂函数，直接求解其满足不等式的连续时间区间是困难的。因此，采用离散时间步进的数值仿真方法来求解，通过步进的方法进行累加求出总时间。

2.2 问题二的分析

问题核心是优化 FY1 运动及烟幕弹投放时序，让单枚烟幕对真目标（形成最长有效遮蔽。为了解决这个高维、非线性、多极值特性，我们决定采用两阶段混合优化策略：，通过将粒子群算法与序列二次规划相结合，利用粒子群算法的全局搜索优势与序列二次规划的局部优化优势，找到最优解。

2.3 问题三的分析

问题三是问题二的推广，烟幕弹数目从 1 枚增至 3 枚。经过分析，我们发现此时会出现同一时间存在多枚烟雾弹的情况。而对于多枚烟幕弹同时存在时的有效遮蔽分析，问题一建立的模型不再完全规范，因此需要对遮蔽判断模型进行修正。于是，需要将圆柱体进行离散化，利用射线投射法，重新建立遮蔽判断模型。

同时，整个优化过程的维度增加，需要增加种群大小与迭代次数，以提高全局搜索能力，避免局部优化。

2.4 问题四的分析

首先可以将 3 个无人机带入问题二的代码中进行分析，求得每一个单独作用时的最大遮蔽时间，三者相加就能得到共同作用时的最大遮蔽时间的上限，当且仅当三个烟幕全程不重叠时取得此最大上限值。

因此，对此时的烟幕进行分析，若确实无遮挡，则直接取得最大值，若存在遮挡，则继续利用混合优化策略，优化得到最大遮挡值。

2.5 问题五的分析

问题五进一步复杂化为多对多协同场景，面临更大挑战。决策变量超过 50 个，属于超高维优化问题，需要同时解决资源分配和参数优化的双重问题，各导弹的飞行特性差异增加了优化难度，计算成本呈指数级增长，对算法效率要求极

高，因此可以考虑简化模型，算出特殊情况之后再进行优化。

三、模型假设

1. 理想运动环境：忽略空气阻力、气流等大气效应对各实体运动轨迹的影响，重力加速度取为 $9.8m^2/s$ 。
2. 瞬时动作：无人机调整方向、投放弹药及弹药起爆过程均视为瞬时完成。
3. 质点与理想几何体：在远距离交战场景中，将导弹、无人机、未爆烟幕弹视为质点；起爆后的烟幕云视为一个维持恒定半径的理想球体。

四、符号说明

符号	说明	单位
O	假目标位置	/
P	真目标位置	/
R	球体半径	m
P_d	投放点位置	/
M_0	导弹初始位置	/
C_b	起爆位置	/
$U_{0,xy}$	无人机初始位置	/
$\vec{d}_{mo}$	无人机初始位置到假目标方向向量	/
v_{mo}	导弹速度	m/s
v_u	无人机行驶速度	m/s
$M(t)$	导弹任意时刻位置	/
$U(t)$	无人机任意时刻位置	/
$C(t)$	烟幕球心位置	/
t_d	干扰弹投放时刻	s
t_f	引信延迟时间	s
d	导弹到烟幕中心距离	m
α	遮蔽锥角	rad
β_0	底面最大视角	rad
β_1	顶面最大视角	rad

注：其他符号已在文章的相应部分给出说明

五、模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 运动学轨迹方程

为精确描述战场态势，首先需为各个实体建立运动学模型。基于模型假设，即忽略空气阻力等次要因素，所有运动均被理想化。

- 导弹轨迹：

导弹从初始位置 $M_1 = (20000, 0, 2000)$ 指向假目标（原点 O ）做匀速直线运动，运动的方向向量为

$$\overrightarrow{d_{mo}} = \frac{\overrightarrow{M_1 O}}{|\overrightarrow{M_1 O}|} \quad (1)$$

导弹的速度大小为 v_{mo} ，故导弹速度的速度向量为： $\overrightarrow{v_{mo}} = v_{mo} \cdot \overrightarrow{d_{mo}}$ 。在任意时刻 t ，其位置 $M(t)$ 可由下式确定：

$$M(t) = M_0 + \overrightarrow{v_m} \cdot t \quad (2)$$

其中 v_{mo} 恒为 300m/s。

- 无人机轨迹：

无人机 FY_j 以恒定速度 v_u 水平飞行，其飞行方向指向一个虚拟目标点 O ， O 作为本场景的坐标原点，无人机水平方向向量为 $\overrightarrow{d_{u,xy}} = \overrightarrow{U_{0,xy} O_{xy}}$ ，无人机水平飞行速度向量：

$$\overrightarrow{v_u} = v_u \cdot \frac{\overrightarrow{d_{u,xy}}}{|\overrightarrow{d_{u,xy}}|} \quad (5)$$

由上式可得无人机在任意时刻 t 的位置：

$$U(t) = U_0 + \overrightarrow{v_u} \cdot t \quad (6)$$

其中 U_0 为无人机初始位置， v_u 为无人机的飞行速度。

- 烟幕弹轨迹：

烟幕投放点为 P_d ，在时刻 t_d ，无人机投下烟幕干扰弹， P_d 的表达式：

$$P_d = U(t_d) = U_0 + \overrightarrow{v_u} \cdot t_d \quad (7)$$

在烟幕干扰投放后，其初始水平速度等于无人机速度 v_u 。经过引信延迟时间后起爆 t_f ，此过程中烟幕干扰弹做平抛运动，期间水平位移 $\Delta p_{xy} = \overrightarrow{v_{u,xy}} \cdot t_f$ ，

垂直位移： $\Delta p_z = -\frac{1}{2}g \cdot t_f^2$ ，因此爆炸点的坐标 C_b 为

$$C_d = P_d + \Delta p_{xy} - (0, 0, \frac{1}{2}gt_f^2) \quad (8)$$

5.1.2 烟幕云动态建模

模型将烟幕云抽象成一个半径为 R 的球体。起爆后 ($t \geq t_b$)，烟雾球随着重力下落球心的位置 $C(t)$ 仅受重力影响，且以恒定下沉速度 $V_s = 3m/s$ ，从起爆点 C_b 处开始垂直下落。到达时刻 t 的烟幕球心位置：

$$C(t) = C_b - [0, 0, v_s \cdot (t - t_b)] \quad (10)$$

5.1.3 有效遮蔽模型

整个过程的尺度十分大，因此我们将导弹抽象为一个质点。遮挡的产生是由于球形的烟幕，挡住了在导弹与目标之间。此时我们可以通过导弹与烟幕的几何关系，算出遮挡的范围，通过判断整个圆柱是否在范围内从而判断是否为有效遮挡。

此时，整个遮挡范围是以导弹为顶点，做一个圆锥使整个烟幕为此圆锥的内切球，此时在这个烟幕内后圆锥内部的集合，就是遮挡范围，称为“遮蔽视锥”。这个“视锥”可以被形象地理解为烟幕云从导弹视角投下的“隐形阴影”。任何完全位于这个锥体内部的物体，对于该导弹而言都是不可见的。

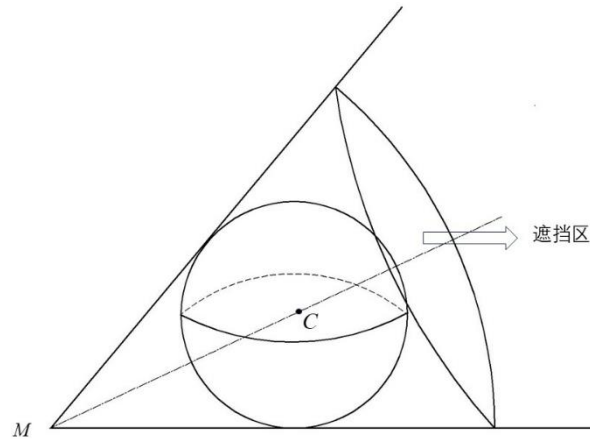


图 1：“视锥”示意图

若要判断点是否在“视锥”内部，我们可以通过“遮蔽角”进行计算。

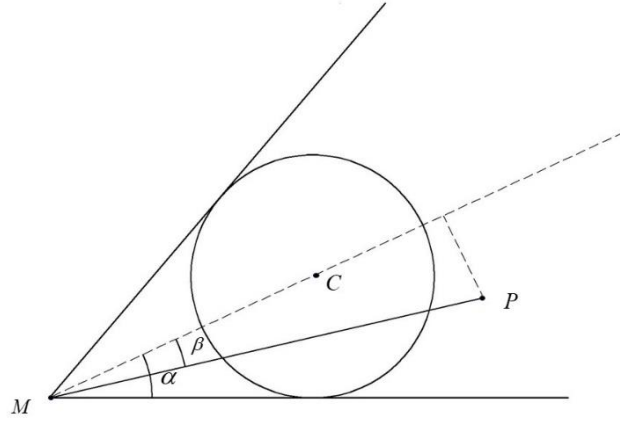


图 2：点在“视锥”内

由于球是内切球，一次“遮蔽角” α 的值能够直接计算：

$$\alpha = \arcsin\left(\frac{R}{d}\right)$$

而点 P 对应的角 β 可以通过向量进行计算：

$$\cos \beta = \frac{\overrightarrow{MP} \cdot \overrightarrow{MC}}{|\overrightarrow{MP}|} \Rightarrow \beta = \arccos\left(\frac{\overrightarrow{MP} \cdot \overrightarrow{MC}}{|\overrightarrow{MP}|}\right)$$

若计算出来的 β 值小于“遮蔽角” α ，则可以确定点 P 在“视锥”内部。

若现在存在两个不同的点 A 、 B ，且 A 、 B 两点均在“视锥”内部，则很容易能够判断出线段 AB 位于“视锥”内部。

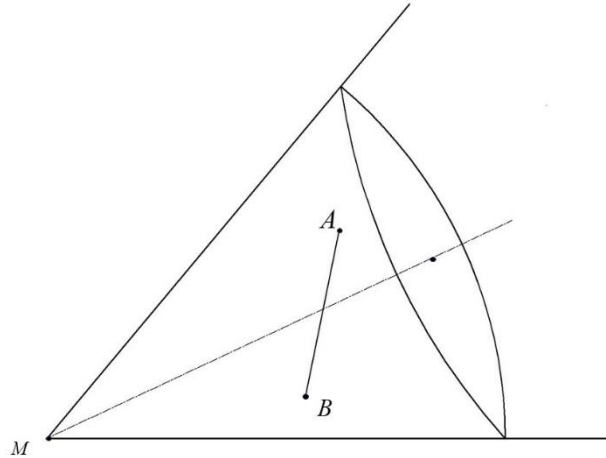


图 3：线段在“视锥”内部

对于一个圆柱形目标，要实现完全遮蔽，必须确保其任何部分都不可见。通过几何分析可以证明一个关键的简化条件：当且仅当圆柱体的上、下两个底面圆周都完全位于“遮蔽视锥”内部时，整个圆柱体即被完全遮蔽。

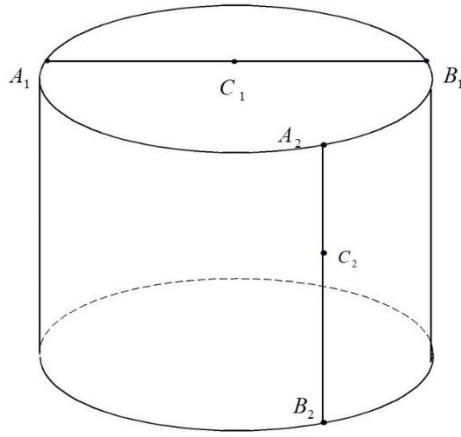


图 4: 圆柱上点与线段关系

这一证明的逻辑链条如下:

1. 圆柱体的表面可以被看作是由连接上、下底面圆周对应点的无数条垂直线构成的。
2. 如果两个点均在凸形的“视锥”内部,那么连接这两点的线段也必然完全位于视锥内部。
3. 因此,只要确保上、下两个圆周上的所有点都在视锥内,那么构成圆柱侧面的所有线段也都在视锥内,从而保证了整个圆柱体的完全遮蔽

这个条件将判断对象从一个三维实体简化为两个二维圆周。要判断一个圆周是否在视锥内,只需找到该圆周上与视锥中心轴夹角最大的点,并检验该点是否在视锥内即可。

5.1.4 模型的计算过程

由于上述角度的瞬时值是随时间变化的复杂函数,直接求解其满足不等式的连续时间区间是困难的。因此,采用离散时间步进的数值仿真方法来求解。

以一个极小的时间步长以 Δt 为时间增量推进仿真。

在任意时 t 刻,我们有导弹的位置 $M(t)$, 烟幕球心位置 $C(t)$ 。

从导弹看向烟幕球,视线与球体相切构成锥体,其半顶角为 α , 导弹到烟幕的距离:

$$d = \left| \overline{M(t)C(t)} \right| \quad (11)$$

根据几何关系,可得:

$$\sin(\alpha) = \frac{R}{d} \quad (12)$$

因此:

$$\alpha = \arcsin\left(\frac{R}{d}\right) \quad (13)$$

从导弹 $M(t)$ 看向目标圆柱体，需找到能包含整个圆柱的最小视锥。其半顶角为 β ，由目标上距离视线中心最远的点决定。定义视线中心轴位向量

$$u = \frac{\overrightarrow{MC}}{d} \quad (14)$$

目标圆柱体由定点圆心 A_1 和底面圆心定义 A_0 。对于圆柱体上任意一点 P ，其视线中心轴的夹角 β 可由点积公式计算：

$$\cos \beta = \frac{\overrightarrow{MP} \cdot \overrightarrow{MC}}{|\overrightarrow{MP}|} \quad (15)$$

圆柱顶面上的点表示为：

$$P_{\perp}(\theta) = A_1 + R(\cos \theta \cdot \vec{x} + \sin \theta \cdot \vec{y}) \quad (16)$$

圆柱底面上的点表示为：

$$P_{\downarrow}(\theta) = A_0 + R(\cos \theta \cdot \vec{x} + \sin \theta \cdot \vec{y}) \quad (17)$$

要找到最大视角 $\beta_{\max}$ ，等价于找到最小的 $\cos(\beta)$ 。

在 t_0 时刻， β_0 表达式：

$$\cos \beta_0 = \frac{\overrightarrow{M(t_0)P_{\downarrow}(\theta)} \cdot \overrightarrow{M(t_0)C(t_0)}}{|\overrightarrow{M(t_0)P_{\downarrow}(\theta)}|} \quad (18)$$

$$\beta_0 = \arccos \frac{\overrightarrow{M(t_0)P_{\downarrow}(\theta)} \cdot \overrightarrow{M(t_0)C(t_0)}}{|\overrightarrow{M(t_0)P_{\downarrow}(\theta)}|} \quad (20)$$

β_1 表达式：

$$\cos \beta_1 = \frac{\overrightarrow{M(t_0)P_{\perp}(\theta)} \cdot \overrightarrow{M(t_0)C(t_0)}}{|\overrightarrow{M(t_0)P_{\perp}(\theta)}|} \quad (21)$$

$$\beta_1 = \arccos \frac{\overrightarrow{M(t_0)P_{\perp}(\theta)} \cdot \overrightarrow{M(t_0)C(t_0)}}{|\overrightarrow{M(t_0)P_{\perp}(\theta)}|} \quad (22)$$

在 t_0 时刻， $M(t_0)$ 与 $C(t_0)$ 的坐标可以直接计算得出，由此可知， $\cos \beta_0$ 、

$\cos \beta_1$ 均为关于 θ 的单变量函数，且 θ 范围已知： $\theta \in [0, 2\pi]$ ，因此可以求出其最大值。但由于直接求导计算稍显困难，本文利用 MATLAB 库中的数值优化方法 $fminbnd$ ，在 $\theta \in [0, 2\pi]$ 的范围内搜寻最小 $\cos \beta$ 值，从而得出最大角度 β 。

在分别计算出上下两圆周的最大角度 β 值后，取二者较大者作为最终的目标视角；

$$\beta_{max} = \max(\beta_0, \beta_1) \quad (23)$$

遮蔽判定当且仅当遮蔽锥角大于等于目标最大视角时，目标完全被遮蔽，遮蔽条件：

$$\alpha \geq \beta_{max} \quad (24)$$

若成功遮蔽，则 $t_{总} = t_{总} + \Delta t$ ($t_{总}$ 初始化为 0)。

5.1.5 结果分析

通过离散时间步进的方法，计算得 FY1 投放单弹对 M1 的有效遮蔽时长为 1.3916s。

8.0565s (有效遮蔽)	x 坐标(m)	y 坐标(m)	z 坐标(m)
导弹	17595.045	0.000	1759.504
烟幕球	17188.000	0.000	1727.627

表一

9.4481s (遮蔽结束)	x 坐标(m)	y 坐标(m)	z 坐标(m)
导弹	17179.637	0.000	1717.964
烟幕球	17188.000	0.000	1723.452

表二

5.2 问题二模型的建立与求解

5.2.1 有效遮蔽模型

此烟幕弹，遮蔽情况仍能视为一个“视锥”模型。对此问题的有效遮蔽模型仍然沿用问题一中的方法。

5.2.2 决策变量

本优化问题包含四个关键决策变量，它们共同决定了烟雾弹的最终起爆位置和时间：

- FY1 的飞行速度：题目中规定 $v_u \in [70, 140] m/s$

- FY1 的飞行方向：导弹始终位在 XZ 平面上运动，无人机 FY1 初始位置同样位于 XZ 平面。若无人机沿着 $\theta \in [\pi, 2\pi] rad$ 即进入 $-Y$ 轴区域，由于真目标位于 $+Y$ 轴，此时必然不能够进行有效遮蔽，得到 $\theta \in [0, \pi] rad$
- 烟幕弹的投放时间：在导弹开始到击中目标过程中均可以进行投放，
 $t_d \in [0, 66.999] s$
- 起爆延时时间：无人机的高度为 1800m，因此对延迟起爆时间也有约束

$$\frac{1}{2} g t_f^2 < 1800s$$

$$t_f < 19.166s$$

若 $t_f \geq 19.166s$ ，则烟幕弹会在接触地面后爆炸，形成的烟雾位于地面，而

此时导弹在上空，烟幕弹无法形成遮蔽效果。得到 $t_f \in [0, 19.166s]$

5.2.3 目标函数

本优化的目标是最大化有效遮蔽时间，即烟幕有效遮蔽目标区域的时间长度。设 $I(t)$ 为指示函数，当目标被有效遮蔽时为 1，否则为 0，最大化有效遮蔽时间：

$$\max T_s = \int_{t_{burst}}^{t_{end}} I(t) dt \quad (25)$$

其中 t_{burst} 为表示烟幕起爆的绝对时间，其表达式：

$$t_{burst} = t_d + t_{fuse} \quad (26)$$

导弹的总体飞行时间 t_{total} 为总出发到击中目标总时间， t_{total} 表达式：

$$t_{total} = \frac{|M_0 O|}{v_m} \quad (27)$$

烟幕有效持续时间 $t_{duration}$ 恒为 20s， t_{end} 表示遮蔽效果计算的时间终点，取烟幕持续时间结束和导弹击中目标时间中的较早者：

$$t_{end} = \min(t_{burst} + t_{duration}, t_{total}) \quad (28)$$

5.2.4 约束条件

决策变量的物理可行范围构成了优化问题的基础约束：

$$s.t. \begin{cases} 70 \leq v_u \leq 140 \\ 0 \leq \theta \leq 2\pi \\ 0 \leq t_d \leq t_{total} \\ 0 \leq t_f \leq t_{total} - t_d \\ U(t) = U_0 + \vec{v}_u \cdot t \\ P_d = U(t_d) \\ P_{burst} = P_d + \vec{v}_u \cdot t_f \\ C(t) = P_{burst} + \vec{v}_s \cdot (t - t_{burst}) \\ M(t) = M_0 + \vec{v}_m \cdot t \cdot \frac{\overrightarrow{M_0 O}}{|\overrightarrow{M_0 O}|} \end{cases} \quad (29)$$

5.2.5 优化方法

针对本问题的高维、非线性、多极值特性，采用两阶段混合优化策略，兼顾全局搜索能力和局部寻优精度。

Step1：全局搜索（粒子群算法 PSO）

粒子群优化算法模拟鸟群社会行为，通过个体与群体经验的平衡来探索解空间。算法中的每个“粒子”代表决策空间中的一个潜在解（即一组特定的无人机飞行和投放参数）。粒子们在解空间中“飞行”，其速度和方向同时受到两个核心因素的影响：它自己历史上的最佳位置和整个种群迄今为止发现的最佳位置。该阶段旨在快速定位，避免陷入局部最优。

Step2：局部精细优化（序列二次规划 SQP）

它通过在当前点将原非线性问题近似为一个二次规划（QP）子问题，并求解该子问题来确定一个最优的前进方向和步长，然后迭代逼近最优解。通过基于 PSO 提供的优质初始解，采用序列二次规划算法进行精细调优：该阶段利用 SQP 算法对梯度信息的有效利用，实现解的高精度优化。



图 5：混合优化策略

5.2.6 实验结果分析

通过 MTALB 代码仿真得到： $v_u = 132.412211\text{m/s}$ ，飞行角度为 5.023246° ，投放时间 $t_d = 0.980950\text{s}$ ，起爆时间延迟 $t_f = 0.000747\text{s}$ ，有效遮蔽时间 4.587s 。

	x 坐标 (m)	y 坐标 (m)	z 坐标 (m)
投放点	17929.3909	11.3731	1800.0000
起爆点	17929.4894	11.3818	1799.9999

表三

通过方向分析，发现无人机与导弹相向飞行，符合实际情况。

5.3 问题三模型的建立与求解

由于此时无人机投放三枚烟幕弹，每个烟幕弹的持续时间为 20s 。经计算导弹击中假目标的总时长约为 66.999s ，有极大可能出现两个烟幕或三个烟幕同时存在的情况，此时，“遮蔽视锥”为所有“视锥”的并集。

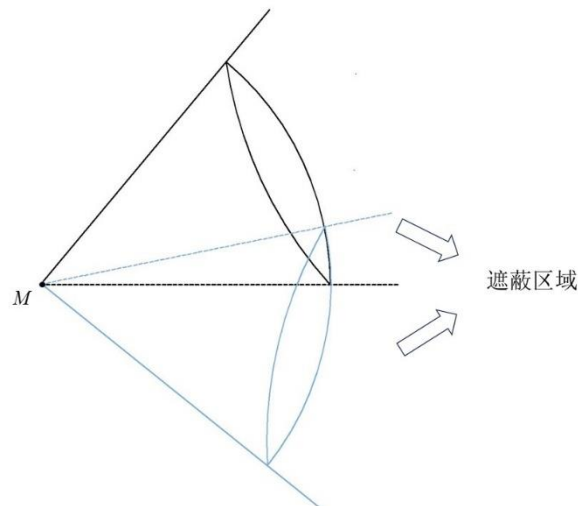


图 4：“视锥”的并集

此时，直接利用视锥的方法将不在便于计算，因此改用 3D 视线算法中的射线投射法进行计算。

5.3.1 射线投射算法

通过观测点向目标点发送射线，检查射线最先击中的是哪个物体。

- 如果射线在击中采样点之前先击中了遮挡物体，则这个目标点不可见。
- 如果射线直接击中了采样点，没有先碰到遮挡物体，则这个采样点可见。

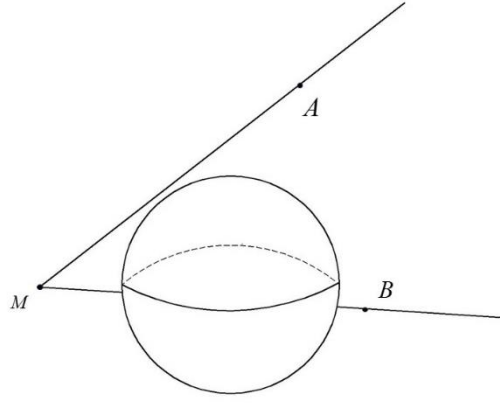


图 5: 射线投射算法

例如图 5 中, M 点为观测点, A 、 B 点为目标点, 球为遮挡物体。以 M 点为端点分别向 A 、 B 。射线 MA 没有碰到遮挡物球, 因此 A 点没有被成功遮蔽; 射线 MB 在击中 B 点之前击中了遮挡物体, 因此 B 点被成功遮蔽。

为判断圆柱是否被完全遮蔽, 从导弹向目标圆柱体表面发射大量离散的视线射线。只有当每一条射线都被至少一个烟幕球阻挡时, 才判定遮蔽成功。

判定步骤如下:

1. 目标表面离散化: 在目标圆柱体表面生成一组密集的采样点。
2. 视线射线构建: 对每个采样点, 构建一条从导弹位置出发的视线射线。
3. 射线-球体求交测试: 对每一条射线, 依次测试它是否与场景中所有的烟幕球相交。若射线与球体存在交点, 且交点位于导弹与目标采样点之间, 则该点被遮蔽。
4. 完全遮蔽判定: 只有当所有的目标采样点都被至少一个烟幕球遮蔽时, 目标在时刻 t 实现完全有效遮蔽。

5.3.2 决策变量

- 无人机飞行速度: $v_u \in [70, 140] m/s$
- 无人机飞行方向: $\theta \in [0, \pi] rad$
- 第一枚烟幕弹投放时间: $t_{d1} \in [0, 64.999] s$
- 第一枚烟幕弹起爆延时: $t_{f1} \in [0, 19.166] s$
- 第二枚烟幕弹投放时间: $t_{d2} \in [t_{d1} + 1, 65.999] s$
- 第二枚烟幕弹起爆延时: $t_{f1} \in [0, 19.166] s$
- 第三枚烟幕弹投放时间: $t_{d3} \in [t_{d2} + 1, 66.999] s$
- 第三枚烟幕弹起爆延时: $t_{f3} \in [0, 19.166] s$

5.3.2 目标函数

最大化三枚烟幕共同形成的有效遮蔽时间:

$$\max T_s = \int_{t_{start}}^{t_{end}} I(t) dt \quad (30)$$

其中为最早起爆时间：

$$t_{start} = \min(t_{burst1}, t_{burst2}, t_{burst3}) \quad (31)$$

第 i 枚烟幕弹起爆的时刻 t_{bursti} 为：

$$t_{bursti} = t_{di} + t_{fi} \quad (32)$$

5.3.3 约束条件

决策变量的物理可行范围构成了优化问题的基础约束：

$$s.t. \begin{cases} 70 \leq v_u \leq 140 \\ 0 \leq \theta \leq \pi \\ 0 \leq t_{d1} \leq t_{total} \\ 0 \leq t_{f1} \leq t_{total} - t_{d1} \\ t_{d1} + 1 \leq t_{d2} \leq t_{total} \\ 0 \leq t_{f2} \leq t_{total} - t_{d2} \\ t_{d2} + 1 \leq t_{d3} \leq t_{total} \\ 0 \leq t_{f3} \leq t_{total} - t_{d3} \\ t_{di} + t_{fi} \leq t_{totali}, i=1,2,3 \\ U(t) = U(0) + \vec{v}_u \cdot t \\ P_{di} = U(t_{di}) \\ P_{bursti} = P_{di} + \vec{v}_u \cdot t_{fi} \\ C_i(t) = P_{bursti} + \vec{v}_s \cdot (t - t_{bursti}), t \geq t_{bursti} \\ M(t) = \vec{M}_0 + \vec{v}_m \cdot t \cdot \frac{\vec{M}_0 O}{|\vec{M}_0 O|} \end{cases} \quad (33)$$

5.3.4 多烟幕弹遮蔽判断条件

多烟幕弹的遮蔽判断采用射线投射法，确保目标圆柱体上的所有采样点都被至少一个烟幕弹遮蔽。

目标圆柱体采样点由以下表达式生成：

$$P_{ij} = A_j + R_i \cdot \left(\cos \theta_i \cdot \vec{x} + \sin \theta_i \cdot \vec{y} \right) \quad (34)$$

其中 A_j 为底面或顶面圆心， θ_i 表达式为：

$$\theta_i = \frac{2\pi i}{N}, i=1,2,\dots,N \quad (35)$$

检测射线到球体的交点，可以列出从导弹位置 $M(t)$ 到采样点 P_{ij} 的射线，参数方程为：

$$R(s) = M(t) + s \cdot \frac{\overrightarrow{M(t)P_{ij}}}{|\overrightarrow{M(t)P_{ij}}|}, 0 \leq s \leq |\overrightarrow{M(t)P_{ij}}| \quad (36)$$

射线与烟幕球体的交点满足，交点到球心的欧几里得距离应等于烟幕球的半径：

$$|\overrightarrow{R(s)C_k(t)}|^2 = R^2 \quad (37)$$

完全遮蔽的条件为 $\forall P_{ij}, \exists k \in \{1,2,3\}$ ，使得 $\overrightarrow{M(t)P_{ij}}$ 所在射线与烟幕球 k 相交。

5.3.4 优化方法

针对此更高维的非线性优化问题，仍然采用 PSO-SQP 混合策略。但是在问题二的基础上适当增加种群大小与迭代次数，以提高全局搜索能力，避免局部优化。

5.3.5 结果与分析

通过模型得到，FY1 以速度 125.3227 m/s、方向角 6.2069° 飞行，有效遮蔽时间为 6.45s。

烟幕干扰弹编号	投放时刻	起爆时刻
1	0.0000	0.0001
2	1.0000	0.0001
3	2.0077	2.0660

表四

烟幕干扰弹编号	投放点 x 坐标 (m)	投放点 y 坐标 (m)	投放点 z 坐标 (m)
1	17800.0000	0.0000	1800.0000
2	17924.5881	13.5496	1800.0000
3	18050.1425	27.2044	1800.0000

表五

烟幕干扰弹编号	起爆点 x 坐标 (m)	起爆点 y 坐标 (m)	起爆点 z 坐标 (m)
1	17800.0124	0.0013	1800.0000
2	17924.6005	13.5510	1800.0000
3	18307.5452	27.2044	1779.0844

表六

5.4 问题四模型的建立与求解

5.4.1 决策变量

- FY1 的飞行速度: $v_{u1} \in [70, 140] m/s$
- FY2 的飞行速度: $v_{u2} \in [70, 140] m/s$
- FY3 的飞行速度: $v_{u3} \in [70, 140] m/s$
- FY1 的飞行方向: $\theta_1 \in [0, \pi] rad$
- FY2 的飞行方向: 由于 FY2 初始位置为(12000, 1400, 1400)，在圆柱体的右后方，而导弹始终在 XZ 平面运动，若想要进行有效遮蔽，FY2 需要靠近 XZ 平面运动，即 $\theta_2 \in [\pi, 2\pi] rad$
- FY3 的飞行方向: 由于 FY3 初始位置为(6000, -3000, 700)，在圆柱体的左后方，而导弹始终在 XZ 平面运动，若想要进行有效遮蔽，FY3 需要靠近 XZ 平面运动，即 $\theta_3 \in [0, \pi] rad$
- FY1 携带烟幕干扰弹投放时间: $t_{d1} \in [0, t_{total}]$
- FY1 携带烟幕干扰弹引信延迟时间: $t_{f1} \in [0, 19.166]$
- FY2 携带烟幕干扰弹投放时间: $t_{d2} \in [0, t_{total}]$
- FY2 携带烟幕干扰弹引信延迟时间: $t_{f2} \in [0, 16.903]$
- FY3 携带烟幕干扰弹投放时间: $t_{d3} \in [0, t_{total}]$
- FY3 携带烟幕干扰弹引信延迟时间: $t_{f3} \in [0, 11.952]$

5.4.3 约束条件

由以上信息和已知条件可构成以下基础约束:

$$s.t \left\{ \begin{array}{l} 70 \leq v_{u1} \leq 140 \\ 70 \leq v_{u2} \leq 140 \\ 70 \leq v_{u3} \leq 140 \\ 0 \leq t_{di} \leq t_{total}, i \in [1, 2, 3] \\ 0 \leq t_{f1} \leq 19.166 \\ 0 \leq t_{f2} \leq 16.903 \\ 0 \leq t_{f3} \leq 11.952 \\ t_{di} + t_{fi} \leq t_{total}, i = 1, 2, 3 \\ U_i(t) = U_i(0) + \vec{v}_{ui} \cdot t \\ P_{di} = U_i(t_{di}) \\ P_{bursti} = P_{di} + \vec{v}_{ui} \cdot t_{fi} \\ C_i(t) = P_{bursti} + \vec{v}_s \cdot (t - t_{bursti}), t \geq t_{bursti} \\ M(t) = \vec{M}_0 + \vec{v}_m \cdot t \cdot \frac{\overrightarrow{M_0 O}}{|\overrightarrow{M_0 O}|} \end{array} \right. \quad (38)$$

5.4.4 计算分析

由于在问题二中已经得到了单架无人机的飞行策略与单枚烟幕弹的投放时序模型，以最大化对单个导弹的遮蔽时长。现在将 $FY2(12000, 1400, 1400)$ 、 $FY3(6000, -3000, 700)$ 分别带入问题二建立的模型中，并直接利用问题二计算得到的 $FY1$ ，得出每一个无人机在投弹一枚情况的最大遮蔽时间。根据代码分析得到：

$FY1$	x 坐标(m)	y 坐标(m)	z 坐标(m)
投放点	17929.3909	11.3731	1800.0000
起爆点	17929.4894	11.3818	1799.9999

表六

- $FY1$: $v_{u1} = 132.4122 \text{ m/s}$ ，飞行角度为 5.0232° ，投放时间 $t_{d1} = 0.9809 \text{ s}$ ，起爆时间延迟 $t_{f1} = 0.000747 \text{ s}$ ，最大有效遮蔽时间 $t_{1\max} = 4.587 \text{ s}$ 。

<i>FY2</i>	x 坐标(m)	y 坐标(m)	z 坐标(m)
投放点	11886.9238	921.0734	1400.0000
起爆点	11682.2329	54.1195	1178.4147

表七

- *FY2*: $v_{u2} = 132.4656 \text{ m/s}$, 飞行角度为 256.7155° , 投放时间 $t_{d2} = 3.7149 \text{ s}$, 起爆时间延迟 $t_{f2} = 6.7247 \text{ s}$, 最大有效遮蔽时间为 $t_{2\max} = 3.867 \text{ s}$ 。

<i>FY3</i>	x 坐标(m)	y 坐标(m)	z 坐标(m)
投放点	6473.4071	-269.6195	700.0000
起爆点	6536.3262	93.2672	663.8598

表八

- *FY3*: $v_{u3} = 135.6144 \text{ m/s}$, 飞行角度为 80.1635° , 投放时间 $t_{d3} = 20.4338 \text{ s}$, 起爆时间延迟 $t_{f3} = 2.7157 \text{ s}$, 最大有效遮蔽时间为 $t_{3\max} = 3.095 \text{ s}$ 。

得到此问最大的遮蔽时间 $t_{\max} \leq t_{1\max} + t_{2\max} + t_{3\max} = 11.549 \text{ s}$, 当且仅当三个烟幕团没有相互重叠时取得最大值。

经观察发现, 三个烟幕的两两距离十分遥远, 且由于烟幕只会垂直下降, 可以计算得到整个过程中, 两个球的球心的最短距离 $d_{ij\min}$:

$$\sqrt{(C_{ix} - C_{jx})^2 + (C_{iy} - C_{jy})^2} \leq d_{ij\min} \quad (39)$$

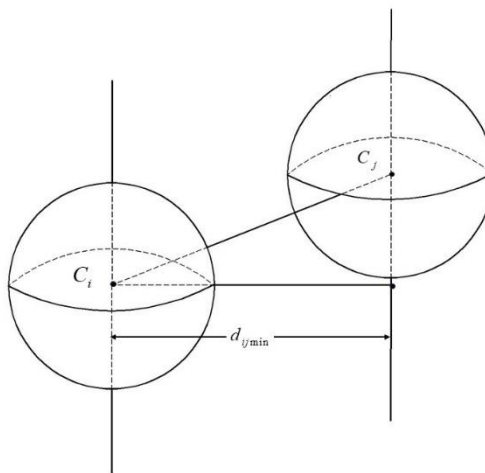


图 5: 射线投射算法

计算求得两两烟幕弹的最小距离:

$$d_{12\min} = \sqrt{(11682.2329 - 17929.4894)^2 + (54.1193 - 11.3818)^2} = 6247.39 \text{ m}$$

$$d_{13\min} = \sqrt{(6546.3262 - 17929.4894)^2 + (93.2672 - 11.3818)^2} = 11383.46 \text{ m}$$

$$d_{23\min} = \sqrt{(6546.3262 - 11682.2329)^2 + (93.2672 - 54.1195)^2} = 5136.0558 \text{ m}$$

发现 $d_{12\min}$ 、 $d_{13\min}$ 、 $d_{23\min}$ 均远大于 2 倍球半径 (20m)，因此无论如何不会出现重叠现象，即 $t_{\max}$ 可以取到最大值 11.549s，不需要再考虑其他的优化情况。

4.5 问题五模型的建立与求解

4.5.1 决策变量

- FYi 的飞行速度: $v_{ui} \in [70, 140] \text{ m/s}, i=1, 2, 3, 4, 5$
- FYi 的飞行方向: $\theta_i \in [0, 2\pi] \text{ rad}$
- FYi 第 1 枚烟幕干扰弹投放时间: $t_{di1} \in [0, t_{total}]$
- FYi 第 1 枚烟幕干扰弹引信延迟时间: $t_{fi1} \in [0, t_{total} - t_{di1}]$
- FYi 第 2 枚烟幕干扰弹投放时间: $t_{di2} \in [t_{di1} + 1, t_{total}]$
- FYi 第 2 枚烟幕干扰弹引信延迟时间: $t_{fi2} \in [0, t_{total} - t_{di2}]$
- FYi 第 3 枚烟幕干扰弹投放时间: $t_{di3} \in [t_{di2} + 1, t_{total}]$
- FYi 第 3 枚烟幕干扰弹引信延迟时间: $t_{fi3} \in [0, t_{total} - t_{di3}]$

5.5.2 约束条件

$$\begin{aligned}
 & \left\{ \begin{array}{l}
 70 \leq v_{ui} \leq 140 \\
 0 \leq t_{di1} \leq t_{total} \\
 t_{di1} + 1 \leq t_{di2} \leq t_{total} \\
 t_{di2} + 1 \leq t_{di3} \leq t_{total} \\
 0 \leq t_{fi1} \leq t_{total} - t_{di1} \\
 0 \leq t_{fi2} \leq t_{total} - t_{di2} \\
 0 \leq t_{fi3} \leq t_{total} - t_{di3} \\
 t_{dij} + t_{fij} \leq t_{total}, i=1, 2, 3, 4, 5, j=1, 2, 3 \\
 s.t. \left\{ \begin{array}{l}
 U_i(t) = U_i(0) + \vec{v}_{ui} \cdot t \\
 P_{dij} = U_i(t_{dij}) \\
 P_{burstij} = P_{dij} + \vec{v}_{ui} \cdot t_{fij} + \frac{1}{2} \vec{g} \cdot t_{fij}^2 \\
 C_i(t) = P_{burstij} + \vec{v}_s \cdot (t - t_{burstij}), t \geq t_{burstij} \\
 M_i(t) = \vec{M}_{i0} + \vec{v}_m \cdot t \cdot \frac{\vec{M}_{i0} \cdot \vec{O}}{|\vec{M}_{i0} \cdot \vec{O}|}
 \end{array} \right.
 \end{array} \right. \quad (40)
 \end{aligned}$$

5.5.2 模型建立

为简化模型，现在只让每个无人机进行分配，只针对单独的导弹进行遮蔽，通过约束条件，采用 PSO-SQP 混合策略并进一步增加种群大小与迭代次

数，以提高全局搜索能力，避免局部优化。

5.5.3 结果分析

通过模型计算可以得到，最长遮蔽时间为 20.40s。

其中无人机 FY1：飞行速度：140.00 m/s，飞行方向： -180.00° 分配导弹：M1，对分配导弹的遮蔽时间：6.95 s。

无人机 FY2：飞行速度：131.90 m/s 飞行方向： -136.96° ，分配导弹：M2，对分配导弹的遮蔽时间：3.65 s。

无人机 FY3：飞行速度：105.78 m/s，飞行方向： 124.41° ，分配导弹：M3，对分配导弹的遮蔽时间：2.40 s。

无人机 FY4：飞行速度：125.66 m/s，飞行方向： -89.97° ，分配导弹：M2，对分配导弹的遮蔽时间：3.75 s。

无人机 FY5：飞行速度：113.21 m/s，飞行方向： 114.91° ，分配导弹：M3，对分配导弹的遮蔽时间：3.65 s。

六、模型整体优缺点评价

6.1 模型优点

(1)通过构建“遮蔽视锥”与“射线投射法”准确的建立了有效遮挡模型，能适用于所有场景。

(2)通过将粒子群算法与序列二次规划相结合，利用粒子群算法的全局搜索优势与序列二次规划的局部优化优势，找到最优解。

(3)通过离散时间步进的方法，解决了连续时间上的问题。可以通过改变步长的方式，取得自己需要的时间精度。

6.2 模型缺点

(1)随着维度的增加，需要增加种群大小与迭代次数，以提高全局搜索能力避免局部优化，但是计算所消耗的时间也会随之增加。

(2)若种群大小与迭代次数不合理，很容易陷入局部最优解。

七、参考文献

- [1] DeepSeek, DeepSeek-V3, 深度求索 (DeepSeek), 2025-09-05
- [2] DeepSeek, DeepSeek-V3, 深度求索 (DeepSeek), 2025-09-06
- [3] 王先超, 韩波, 张开银, 等. 粒子群算法在求解数学建模最优化问题中的应用 [J]. 阜 阳 师 范 学 院 学 报 (自 然 科 学 版), 2016, 33(02):117-121. DOI:10.14096/j.cnki.cn34-1069/n/1004-4329(2016)02-117-05.
- [4] 雷开友. 粒子群算法及其应用研究[D]. 西南大学, 2006.

附录

附录 1

支撑材料

1. Q1.m
2. Q2.m
3. Q3.m
4. Q4.m
5. Q5.m
6. result1.xlsx
7. result2.xlsx
8. result3.xlsx
9. 演示文稿 2.jpg
10. 演示文稿 3.jpg
11. 演示文稿 4.jpg
12. 演示文稿 5.jpg
13. 演示文稿 6.jpg
14. 演示文稿 7.jpg
15. 演示文稿 8.jpg
16. 演示文稿 9.jpg
17. 演示文稿 10.jpg
18. AI 工具使用详情.pdf

附录 2

问题一

```
function Q1()
    g = 9.8;
    v_m = 300;
    M1_initial = [20000, 0, 2000];
    T_fake_target = [0, 0, 0];
    P_target_center = [0, 200, 0];
    r_target = 7;
    h_target = 10;
    R_smoke = 10;
    v_sink = 3;
    smoke_duration = 20;
    uav_pos_initial = [17800, 0, 1800];
    uav_speed = 120;
    horizontal_dir_xy = T_fake_target(1:2) - uav_pos_initial(1:2);
    uav_vel = [uav_speed * (horizontal_dir_xy / norm(horizontal_dir_xy)), 0];
    t_drop_delay = 1.5;
    t_fuse_delay = 3.6;
```

```

total_shaded_time = 0;
t_drop = t_drop_delay;
p_drop = uav_pos_initial + uav_vel * t_drop;
v_drop = uav_vel;
t_burst = t_drop + t_fuse_delay;
C_burst(1) = p_drop(1) + v_drop(1) * t_fuse_delay;
C_burst(2) = p_drop(2) + v_drop(2) * t_fuse_delay;
C_burst(3) = p_drop(3) + v_drop(3) * t_fuse_delay - 0.5 * g * t_fuse_delay^2;
A0 = [P_target_center(1), P_target_center(2), 0];
A1 = [P_target_center(1), P_target_center(2), h_target];
for t = t_burst : dt : (t_burst + smoke_duration)
    dist_to_target_total = norm(M1_initial - T_fake_target);
    dir_missile = (T_fake_target - M1_initial) / dist_to_target_total;
    M_t = M1_initial + dir_missile * v_m * t;

    time_to_impact = dist_to_target_total / v_m;
    if t >= time_to_impact, break; end
    time_after_burst = t - t_burst;
    C_t = C_burst - [0, 0, v_sink * time_after_burst];
    is_covered = is_cylindrical_target_occluded_numeric(M_t, C_t, R_smoke, A0, A1,
r_target);

    if is_covered
        total_shaded_time = total_shaded_time + dt;
    end
end
end
function is_occluded = is_cylindrical_target_occluded_numeric(M, C, R_smoke, A0, A1,
R_target)
    MC = C - M;
    d = norm(MC);
    if d < 1e-9 || R_smoke >= d
        is_occluded = true;
        return;
    end
    % 计算圆锥半角  $\alpha$ 
    sin_alpha = R_smoke / d;
    alpha = asin(sin_alpha);
    u = MC / d;
    v = [1, 0, 0];
    w = [0, 1, 0];
    beta0 = calculate_max_angle_numeric(M, A0, u, v, w, R_target);

```



```

    beta1 = calculate_max_angle_numeric(M, A1, u, v, w, R_target);
    max_beta = max(beta0, beta1);
    is_occluded = (max_beta <= alpha);
end
function beta = calculate_max_angle_numeric(M, A, u, v, w, R_target)
    function cos_beta = objective_func(theta)
        P = A + R_target * (cos(theta) * v + sin(theta) * w);
        % 计算向量 MP
        MP = P - M;
        % 计算 cosβ
        cos_beta = dot(MP, u) / norm(MP);
    end
    % 使用 fminbnd 寻找使 cosβ 最小化的 θ
    options = optimset('Display', 'off', 'TolX', 1e-6);
    [theta_opt, min_cos_beta] = fminbnd(@objective_func, 0, 2*pi, options);
    % 计算最大夹角 β
    beta = acos(min_cos_beta);
end

```

附录 3

问题二

```

function main()
    % 定义常量
    g = 9.8;
    v_m = 300;
    M1_initial = [20000, 0, 2000];
    T_fake_target = [0, 0, 0];
    P_target_center = [0, 200, 0];
    r_target = 7;
    h_target = 10;
    R_smoke = 10;
    v_sink = 3;
    smoke_duration = 20;
    uav_pos_initial = [17800, 0, 1800];

    % 计算导弹飞行总时间
    dist_to_target = norm(M1_initial - T_fake_target);
    time_total = dist_to_target / v_m;

    % 优化变量边界

```

```

lb = [70, 0, 0, 0]; % v_uav_min, theta_min, t_drop_min, t_fuse_delay_min
ub = [140, pi, time_total, 20]; % v_uav_max, theta_max, t_drop_max, t_fuse_delay_max

% 第一阶段：全局搜索（使用 PSO）
options_pso = optimoptions('particleswarm', ...
    'SwarmSize', 100, ...
    'MaxIterations', 200, ...
    'Display', 'iter', ...
    'UseVectorized', false, ...
    'FunctionTolerance', 1e-3, ...
    'SelfAdjustmentWeight', 1.49, ...
    'SocialAdjustmentWeight', 1.49, ...
    'InertiaRange', [0.1, 1.1]);

[x_pso, fval_pso] = particleswarm(@(x) high_precision_objective(x, g, v_m, M1_initial,
T_fake_target, P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration,
uav_pos_initial), ...
    4, lb, ub, options_pso);

% 第二阶段：局部精细优化（使用 fmincon）
options_fmincon = optimoptions('fmincon', ...
    'Display', 'iter', ...
    'Algorithm', 'sqp', ...
    'MaxFunctionEvaluations', 1000, ...
    'FunctionTolerance', 1e-6, ...
    'StepTolerance', 1e-6, ...
    'OptimalityTolerance', 1e-6);

[x_opt, fval] = fmincon(@(x) high_precision_objective(x, g, v_m, M1_initial,
T_fake_target, P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration,
uav_pos_initial), ...
    x_pso, [], [], [], [], lb, ub, [], options_fmincon);

% 提取最优解
v_uav_opt = x_opt(1);
theta_opt = x_opt(2);
t_drop_opt = x_opt(3);
t_fuse_delay_opt = x_opt(4);

% 计算投放点和起爆点
uav_vel_opt = [v_uav_opt * cos(theta_opt), v_uav_opt * sin(theta_opt), 0];
p_drop_opt = uav_pos_initial + uav_vel_opt * t_drop_opt;

```

```

    p_burst_x = p_drop_opt(1) + uav_vel_opt(1) * t_fuse_delay_opt;
    p_burst_y = p_drop_opt(2) + uav_vel_opt(2) * t_fuse_delay_opt;
    p_burst_z = p_drop_opt(3) + uav_vel_opt(3) * t_fuse_delay_opt - 0.5 * g *
t_fuse_delay_opt^2;
    p_burst_opt = [p_burst_x, p_burst_y, p_burst_z];

    % 计算有效遮蔽时间
    shaded_time = -fval;

    % 验证结果
    final_shaded_time = verify_solution(v_uav_opt, theta_opt, t_drop_opt,
t_fuse_delay_opt, ...
        g, v_m, M1_initial, T_fake_target, P_target_center, r_target, h_target, ...
        R_smoke, v_sink, smoke_duration, uav_pos_initial);

end

% 高精度目标函数
function f = high_precision_objective(x, g, v_m, M1_initial, T_fake_target, P_target_center,
r_target, h_target, R_smoke, v_sink, smoke_duration, uav_pos_initial)
    % 提取变量
    v_uav = x(1);
    theta = x(2);
    t_drop = x(3);
    t_fuse_delay = x(4);

    % 无人机速度向量
    uav_vel = [v_uav * cos(theta), v_uav * sin(theta), 0];

    % 计算投放点
    p_drop = uav_pos_initial + uav_vel * t_drop;

    % 计算起爆点
    t_burst = t_drop + t_fuse_delay;
    C_burst(1) = p_drop(1) + uav_vel(1) * t_fuse_delay;
    C_burst(2) = p_drop(2) + uav_vel(2) * t_fuse_delay;
    C_burst(3) = p_drop(3) + uav_vel(3) * t_fuse_delay - 0.5 * g * t_fuse_delay^2;

    % 计算导弹飞行总时间
    dist_to_target = norm(M1_initial - T_fake_target);
    time_total = dist_to_target / v_m;

```

```

% 定义目标圆柱体的顶底圆心
A0 = [P_target_center(1), P_target_center(2), 0];
A1 = [P_target_center(1), P_target_center(2), h_target];

% 高精度模拟参数
dt = 0.01; % 时间步长
total_shaded_time = 0;

% 模拟从起爆时间到起爆后 20 秒或导弹击中目标
t_start = t_burst;
t_end = min(t_burst + smoke_duration, time_total);
t_sim = t_start:dt:t_end;

% 导弹方向向量
dir_missile = (T_fake_target - M1_initial) / dist_to_target;

% 并行计算（如果可用）
if isempty(gcp('nocreate'))
    parpool('local');
end

parfor i = 1:length(t_sim)
    t = t_sim(i);

    % 计算导弹位置
    M_t = M1_initial + dir_missile * v_m * t;

    % 计算烟幕位置（匀速下沉）
    time_after_burst = t - t_burst;
    C_t = C_burst - [0, 0, v_sink * time_after_burst];

    % 数值优化方法遮蔽检查
    is_covered = is_cylindrical_target_occluded_numeric(M_t, C_t, R_smoke, A0, A1,
r_target);

    if is_covered
        total_shaded_time = total_shaded_time + dt;
    end
end

% 返回负的遮蔽时间
f = -total_shaded_time;

```

```

end

% 数值优化方法遮蔽判断函数
function is_occluded = is_cylindrical_target_occluded_numeric(M, C, R_smoke, A0, A1,
R_target)
    % 计算导弹到烟雾中心的距离
    MC = C - M;
    d = norm(MC);

    % 特殊情况处理
    if d < 1e-9 || R_smoke >= d
        is_occluded = true;
        return;
    end

    % 计算圆锥半角  $\alpha$ 
    sin_alpha = R_smoke / d;
    alpha = asin(sin_alpha);

    % 计算 MC 的单位向量
    u = MC / d;

    % 使用 x 和 y 轴的单位向量作为 v 和 w
    v = [1, 0, 0];
    w = [0, 1, 0];

    % 计算下顶面的最大夹角
    beta0 = calculate_max_angle_numeric(M, A0, u, v, w, R_target);

    % 计算上顶面的最大夹角
    beta1 = calculate_max_angle_numeric(M, A1, u, v, w, R_target);

    % 取最大夹角
    max_beta = max(beta0, beta1);

    % 判断是否完全遮蔽
    is_occluded = (max_beta <= alpha);
end

function beta = calculate_max_angle_numeric(M, A, u, v, w, R_target)
    % 定义目标函数：计算给定  $\theta$  时的  $\cos\beta$ 
    function cos_beta = objective_func(theta)

```

```

        % 计算圆柱顶面上的点 P
        P = A + R_target * (cos(theta) * v + sin(theta) * w);
        % 计算向量 MP
        MP = P - M
        % 计算  $\cos\beta$ 
        cos_beta = dot(MP, u) / norm(MP);
    end
    % 使用 fminbnd 寻找使  $\cos\beta$  最小化的  $\theta$ 
    options = optimset('Display', 'off', 'TolX', 1e-6);
    [theta_opt, min_cos_beta] = fminbnd(@objective_func, 0, 2*pi, options);
    % 计算最大夹角  $\beta$ 
    beta = acos(min_cos_beta);
end

function shaded_time = verify_solution(v_uav, theta, t_drop, t_fuse_delay, g, v_m, M1_initial,
T_fake_target, P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration,
uav_pos_initial)
    uav_vel = [v_uav * cos(theta), v_uav * sin(theta), 0];
    p_drop = uav_pos_initial + uav_vel * t_drop;
    t_burst = t_drop + t_fuse_delay;
    C_burst(1) = p_drop(1) + uav_vel(1) * t_fuse_delay;
    C_burst(2) = p_drop(2) + uav_vel(2) * t_fuse_delay;
    C_burst(3) = p_drop(3) + uav_vel(3) * t_fuse_delay - 0.5 * g * t_fuse_delay^2;
    dist_to_target = norm(M1_initial - T_fake_target);
    time_total = dist_to_target / v_m;
    A0 = [P_target_center(1), P_target_center(2), 0];
    A1 = [P_target_center(1), P_target_center(2), h_target];
    dt = 0.001; % 时间步长
    total_shaded_time = 0;
    t_start = t_burst;
    t_end = min(t_burst + smoke_duration, time_total);
    t_sim = t_start:dt:t_end;
    dir_missile = (T_fake_target - M1_initial) / dist_to_target;
    for t = t_sim
        M_t = M1_initial + dir_missile * v_m * t;
        time_after_burst = t - t_burst;
        C_t = C_burst - [0, 0, v_sink * time_after_burst];
        is_covered = is_cylindrical_target_occluded_numeric(M_t, C_t, R_smoke, A0, A1,
r_target);
        if is_covered
            total_shaded_time = total_shaded_time + dt;
        end
    end
end

```

```

        shaded_time = total_shaded_time;
    end

```

附录 4

问题三

```

function main()
    g = 9.8;
    v_m = 300;
    M1_initial = [20000, 0, 2000];
    T_fake_target = [0, 0, 0];
    P_target_center = [0, 200, 0];
    r_target = 7;
    h_target = 10;
    R_smoke = 10;
    v_sink = 3;
    smoke_duration = 20;
    uav_pos_initial = [17800, 0, 1800];
    min_drop_interval = 1;
    dist_to_target = norm(M1_initial - T_fake_target);
    time_total = dist_to_target / v_m;
    % 导弹方向向量
    dir_missile = (T_fake_target - M1_initial) / dist_to_target;
    % 优化变量边界
    % x = [v_uav, theta, t_drop1, t_fuse_delay1, t_drop2, t_fuse_delay2, t_drop3,
t_fuse_delay3]
    lb = [70, 0, 0, min_fuse_delay, min_drop_interval, min_fuse_delay, min_drop_interval*2,
min_fuse_delay];
    ub = [140, pi, time_total*0.8, 10, time_total*0.8, 10, time_total*0.8, 10];

    %全局搜索
    options_pso = optimoptions('particleswarm', ...
        'SwarmSize', 500, ...
        'MaxIterations', 1000, ...
        'Display', 'iter', ...
        'UseVectorized', false, ...
        'FunctionTolerance', 1e-3, ...
        'SelfAdjustmentWeight', 1.49, ...
        'SocialAdjustmentWeight', 1.49, ...
        'InertiaRange', [0.5, 1.1]);

```

```

[x_pso, fval_pso] = particleswarm(@(x) multi_smoke_objective(x, g, v_m, M1_initial,
T_fake_target, P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration,
uav_pos_initial, min_drop_interval, time_total, dir_missile), ...
8, lb, ub, options_pso);

% 第二阶段：局部精细优化
options_fmincon = optimoptions('fmincon', ...
'Display', 'iter', ...
'Algorithm', 'sqp', ...
'MaxFunctionEvaluations', 3000, ...
'FunctionTolerance', 1e-6, ...
'StepTolerance', 1e-6, ...
'OptimalityTolerance', 1e-6);

% 添加非线性约束确保投放间隔和时间总约束
nonlcon = @(x) combined_constraint(x, min_drop_interval, time_total, min_fuse_delay);

[x_opt, fval] = fmincon(@(x) multi_smoke_objective(x, g, v_m, M1_initial,
T_fake_target, P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration,
uav_pos_initial, min_drop_interval, time_total, dir_missile), ...
x_pso, [], [], [], [], lb, ub, nonlcon, options_fmincon);

% 提取最优解
v_uav_opt = x_opt(1);
theta_opt = x_opt(2);
t_drop1 = x_opt(3);
t_fuse_delay1 = x_opt(4);
t_drop2 = x_opt(5);
t_fuse_delay2 = x_opt(6);
t_drop3 = x_opt(7);
t_fuse_delay3 = x_opt(8);

uav_vel_opt = [v_uav_opt * cos(theta_opt), v_uav_opt * sin(theta_opt), 0];

p_drop1 = uav_pos_initial + uav_vel_opt * t_drop1;
p_burst1 = calculate_burst_point(p_drop1, uav_vel_opt, t_fuse_delay1, g);

p_drop2 = uav_pos_initial + uav_vel_opt * t_drop2;
p_burst2 = calculate_burst_point(p_drop2, uav_vel_opt, t_fuse_delay2, g);

p_drop3 = uav_pos_initial + uav_vel_opt * t_drop3;
p_burst3 = calculate_burst_point(p_drop3, uav_vel_opt, t_fuse_delay3, g);

```



```

% 计算有效遮蔽时间
shaded_time = -fval;
% 验证导弹在几个关键时间点的位置
t_check = [0, time_total/2, time_total];
for i = 1:length(t_check)
    t = t_check(i);
    M_t = M1_initial + dir_missile * v_m * t;
end
end
function p_burst = calculate_burst_point(p_drop, uav_vel, t_fuse_delay, g)
    p_burst_x = p_drop(1) + uav_vel(1) * t_fuse_delay;
    p_burst_y = p_drop(2) + uav_vel(2) * t_fuse_delay;
    p_burst_z = p_drop(3) + uav_vel(3) * t_fuse_delay - 0.5 * g * t_fuse_delay^2;
    p_burst = [p_burst_x, p_burst_y, p_burst_z];
end
function [c, ceq] = combined_constraint(x, min_interval, time_total, min_fuse_delay)
    % 投放间隔约束
    c1 = [x(3) + min_interval - x(5);    % t_drop1 + min_interval <= t_drop2
          x(5) + min_interval - x(7)];    % t_drop2 + min_interval <= t_drop3
    : t_drop_i + t_fuse_delay_i <= time_total for i=1,2,3
    c2 = [x(3) + x(4) - time_total;
          x(5) + x(6) - time_total;
          x(7) + x(8) - time_total];
    c3 = [min_fuse_delay - x(4);
          min_fuse_delay - x(6);
          min_fuse_delay - x(8)];
    c = [c1; c2; c3];
    ceq = [];
end
function f = multi_smoke_objective(x, g, v_m, M1_initial, T_fake_target, P_target_center,
r_target, h_target, R_smoke, v_sink, smoke_duration, uav_pos_initial, min_drop_interval,
time_total, dir_missile)
    % 提取变量
    v_uav = x(1);
    theta = x(2);
    t_drop1 = x(3);
    t_fuse_delay1 = x(4);
    t_drop2 = x(5);
    t_fuse_delay2 = x(6);
    t_drop3 = x(7);
    t_fuse_delay3 = x(8);

```

```

% 检查投放间隔约束和时间总约束
if t_drop2 < t_drop1 + min_drop_interval || t_drop3 < t_drop2 + min_drop_interval || ...
    t_drop1 + t_fuse_delay1 > time_total || t_drop2 + t_fuse_delay2 > time_total || t_drop3
+ t_fuse_delay3 > time_total
    return;
end
uav_vel = [v_uav * cos(theta), v_uav * sin(theta), 0];
p_drop1 = uav_pos_initial + uav_vel * t_drop1;
p_burst1 = calculate_burst_point(p_drop1, uav_vel, t_fuse_delay1, g);
p_drop2 = uav_pos_initial + uav_vel * t_drop2;
p_burst2 = calculate_burst_point(p_drop2, uav_vel, t_fuse_delay2, g);
p_drop3 = uav_pos_initial + uav_vel * t_drop3;
p_burst3 = calculate_burst_point(p_drop3, uav_vel, t_fuse_delay3, g);
A0 = [P_target_center(1), P_target_center(2), 0];
A1 = [P_target_center(1), P_target_center(2), h_target];
sample_points = generate_sample_points(P_target_center, r_target, h_target);
dt = 0.05; % 时间步长
total_shaded_time = 0;
t_burs1 = t_drop1 + t_fuse_dela1;
t_burs2 = t_drop2 + t_fuse_dela2;
t_burs3 = t_drop3 + t_fuse_dela3;
t_start = min([t_burs1, t_burs2, t_burt3]);
t_end = min(max([t_burs1, t_burt2, t_burs3]) + smoke_duration, time_total);
t_sim = t_star:dt:t_end;

% 初始化烟幕弹位置数组（用于惩罚项计算）
smoke_positions_for_penalty = [];

% 计算烟幕弹在起爆时刻的位置（用于惩罚项）
smoke_positions_for_penalty = [smoke_positions_for_pealty; p_burst1];
smoke_positions_for_penalty = [smoke_positions_for_penlty; p_burst2];
smoke_positions_for_penalty = [smoke_positions_for_penlty; p_burst3];

% 并行计算（如果可用）
if isempty(gcp('nocreate'))
    prpool('local');
end

parfor i = 1:length(t_sim)
    t = t_sim(i);
    M_t = M1_initial + dir_missile * v_m * t;

```

```

smok_positions = [];
if t >= t_burst1 && t <= t_burst1 + smoke_duration
    time_after_burst1 = t - t_burst1;
    C_t1 = p_burst1 - [0, 0, v_sink * time_after_burst1];
    smoke_positions = [smoke_positions; C_t1];
end
if t >= t_burst2 && t <= t_burst2 + smoke_duration
    time_after_burst2 = t - t_burst2;
    C_t2 = p_burst2 - [0, 0, v_sink * time_after_burst2];
    smoke_positions = [smoke_positions; C_t2];
end
if t >= t_burst3 && t <= t_burst3 + smoke_duration
    time_after_burst3 = t - t_burst3;
    C_t3 = p_burst3 - [0, 0, v_sink * time_after_burst3];
    smoke_positions = [smoke_positions; C_t3];
end
if isempty(smoke_positions)
    continue;
end
is_covered = is_target_fully_covered(M_t, smoke_positions, R_smoke,
sample_points);

if is_covered
    total_shaded_time = total_shaded_time + dt;
end
end
% 添加惩罚项
penalty = 0;
for t = [t_burst1, t_burst2, t_burst3]
    M_t = M1_initial + dir_missile * v_m * t;
    for j = 1:size(smoke_positions_for_penalty, 1)
        C = smoke_positions_for_penalty(j, :);
        relative_pos = C - M_t;
        if dot(relative_pos, -dir_missile) > 0
            penalty = penalty + 100;
        end
    end
end
f = -total_shaded_time + penalty;
end
function points = generate_sample_points(center, radius, height)
points = [];

```

```

points = [points; center(1), center(2), 0];
points = [points; center(1), center(2), height];
angles = 0:pi/4:2*pi;
for i = 1:length(angles)-1
    angle = angles(i);
    x = center(1) + radius * cos(angle);
    y = center(2) + radius * sin(angle);
    points = [points; x, y, 0];
    points = [points; x, y, height];
end
for i = 1:length(angles)-1
    angle = angles(i);
    x = center(1) + radius * cos(angle);
    y = center(2) + radius * sin(angle);

    points = [points; x, y, height/2];
end
end
% 检查完全遮蔽
function is_covered = is_target_fully_covered(M, smoke_centers, R_smoke, sample_points)
    % 对每个采样点检查是否被至少一个烟幕弹遮蔽
    for i = 1:size(sample_points, 1)
        P = sample_points(i, :);
        ray_origin = M;
        ray_direction = P - M;
        ray_length = norm(ray_direction);
        ray_direction = ray_direction / ray_length;

        point_covered = false;

        % 检查每个烟幕弹
        for j = 1:size(smoke_centers, 1)
            C = smoke_centers(j, :);
            % 计算射线与球体的交点
            [intersect, t] = ray_sphere_intersection(ray_origin, ray_direction, C, R_smoke);
            % 如果射线与球体相交，并且交点在导弹和目标点之间
            if intersect && t > 0 && t < ray_length
                point_covered = true;
                break;
            end
        end
    end
    % 如果有一个点没有被遮蔽，则目标没有被完全遮蔽
end

```

```

        if ~point_covered
            is_covered = false;
            return;
        end
    end
    is_covered = true;
end

% 计算射线与球体的交点
function [intersect, t] = ray_sphere_intersection(ray_origin, ray_direction, sphere_center,
sphere_radius)
    % 向量从射线原点到球心
    oc = ray_origin - sphere_center;
    a = dot(ray_direction, ray_direction);
    b = 2 * dot(oc, ray_direction);
    c = dot(oc, oc) - sphere_radius^2;
    discriminant = b^2 - 4*a*c;
    if discriminant < 0
        % 没有实根, 没有交点
        intersect = false;
        t = inf;
    else
        % 计算交点参数
        t1 = (-b - sqrt(discriminant)) / (2*a);
        t2 = (-b + sqrt(discriminant)) / (2*a);
        if t1 >= 0 && t1 <= t2
            t = t1;
            intersect = true;
        elseif t2 >= 0
            t = t2;
            intersect = true;
        else
            % 两个交点都在射线后方
            intersect = false;
            t = inf;
        end
    end
end

function shaded_time = verify_multi_smoke_solution(v_uav, theta, t_drops, t_fuse_delays, g,
v_m, M1_initial, T_fake_target, P_target_center, r_target, h_target, R_smoke, v_sink,
smoke_duration, uav_pos_initial)

```

```

dist_to_target = norm(M1_initial - T_fake_target);
time_total = dist_to_target / v_m;
dir_missile = (T_fake_target - M1_initial) / dist_to_target;
A0 = [P_target_center(1), P_target_center(2), 0];
A1 = [P_target_center(1), P_target_center(2), h_target];
sample_points = generate_sample_points(P_target_center, r_target, h_target);
dt = 0.01;
total_shaded_time = 0;

% 计算三个烟幕弹的起爆时间
t_burst1 = t_drops(1) + t_fuse_delays(1);
t_burst2 = t_drops(2) + t_fuse_delays(2);
t_burst3 = t_drops(3) + t_fuse_delays(3);
t_start = min([t_burst1, t_burst2, t_burst3]);
t_end = min(max([t_burst1, t_burst2, t_burst3]) + smoke_duration, time_total);
t_sim = t_start:dt:t_end;
% 使用射线投射方法的遮蔽判断
for i = 1:length(t_sim)
    t = t_sim(i);
    M_t = M1_initial + dir_missile * v_m * t;

    % 计算三个烟幕弹的位置（如果已经起爆）
    smoke_positions = [];
    if t >= t_burst1 && t <= t_burst1 + smoke_duration
        time_after_burst1 = t - t_burst1;
        C_t1 = p_burst1 - [0, 0, v_sink * time_after_burst1];
        smoke_positions = [smoke_positions; C_t1];
    end
    if t >= t_burst2 && t <= t_burst2 + smoke_duration
        time_after_burst2 = t - t_burst2;
        C_t2 = p_burst2 - [0, 0, v_sink * time_after_burst2];
        smoke_positions = [smoke_positions; C_t2];
    end
    if t >= t_burst3 && t <= t_burst3 + smoke_duration
        time_after_burst3 = t - t_burst3;
        C_t3 = p_burst3 - [0, 0, v_sink * time_after_burst3];
        smoke_positions = [smoke_positions; C_t3];
    end
    % 如果没有烟幕弹存在，跳过
    if isempty(smoke_positions)
        continue;
    end
end

```

```

        % 检查目标是否被完全遮蔽
        is_covered = is_target_fully_covered(M_t, smoke_positions, R_smoke,
sample_points);

        if is_covered
            total_shaded_time = total_shaded_time + dt;
        end
    end
    shaded_time = total_shaded_time;
end

```

附录 5

问题五

% 多无人机多导弹烟幕干扰策略优化 - 最终完整版本

```
function final_complete_multi_uav_smoke_strategy()
```

```

    g = 9.8;
    v_missile = 300;
    R_smoke = 10;
    v_sink = 3;
    smoke_duration = 20;
    min_drop_interval = 1;
    min_fuse_delay = 0.1;
    T_fake_target = [0, 0, 0];
    P_target_center = [0, 200, 0];
    r_target = 7; %
    h_target = 10; %
    M1_initial = [20000, 0, 2000];
    M2_initial = [19000, 600, 2100];
    M3_initial = [18000, -600, 1900];
    missiles = {M1_initial, M2_initial, M3_initial};
    UAV_positions = {
        [17800, 0, 1800], % FY1
        [12000, 1400, 1400], % FY2
    }

```

```

        [6000, -3000, 700], % FY3
        [11000, 2000, 1800], % FY4
        [13000, -2000, 1300] % FY5
    };
    missile_dirs = cell(1, 3);
    missile_times = zeros(1, 3);
    for i = 1:3
        dist = norm(missiles{i} - T_fake_target);
        missile_dirs{i} = (T_fake_target - missiles{i}) / dist;
        missile_times(i) = dist / v_missile;
    end
    uav_assignments = {1, 2, 3, 2, 3}; % FY1->M1, FY2->M2, FY3->M3, FY4->M2,
FY5->M3

    for uav_id = 1:5
        fprintf('无人机 FY%d 分配导弹: M%d\n', uav_id, uav_assignments{uav_id});
    end
    results = cell(1, 5);
    for uav_id = 1:5
        fprintf('\n 优化无人机 FY%d 的策略...\n', uav_id);
        assigned_missile = uav_assignments{uav_id};
        results{uav_id} = optimize_uav_strategy_final(...
            UAV_positions{uav_id}, assigned_missile, missile_dirs{assigned_missile},
missiles{assigned_missile}, ...
            T_fake_target, P_target_center, r_target, h_target, R_smoke, v_sink,
smoke_duration, ...
            g, v_missile, min_drop_interval, min_fuse_delay,
missile_times(assigned_missile));
    end
    overall_effectiveness = evaluate_simultaneous_coverage_final(...
        results, missiles, missile_dirs, missile_times, ...
        P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration, g, v_missile);

```



```

end
function result = optimize_uav_strategy_final(...
    uav_initial, assigned_missile, missile_dir, missile_initial, ...
    T_fake_target, P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration, ...
    g, v_missile, min_drop_interval, min_fuse_delay, missile_time)

% 设置优化选项 - 增加迭代次数和种群大小
options = optimoptions('particleswarm', ...
    'SwarmSize', 100, ...
    'MaxIterations', 300, ...
    'Display', 'iter', ...
    'UseVectorized', false);

% 变量: [v_uav, theta, t_drop1, t_fuse1, t_drop2, t_fuse2, t_drop3, t_fuse3]
dist_to_target = norm(missile_initial - T_fake_target);
optimal_drop_time = dist_to_target / v_missile * 0.7; % 在导弹到达前 30%时间投放

lb = [70, -pi, 0, min_fuse_delay, min_drop_interval, min_fuse_delay,
min_drop_interval*2, min_fuse_delay];
ub = [140, pi, missile_time*0.9, 15, missile_time*0.9, 15, missile_time*0.9, 15];
[x_opt, fval] = particleswarm(...
    @(x) uav_objective_function_final(x, uav_initial, missile_dir, missile_initial, ...
    T_fake_target, P_target_center, r_target, h_target, R_smoke, v_sink, ...
    smoke_duration, g, v_missile, min_drop_interval, missile_time), ...
    8, lb, ub, options);
result.v_uav = x_opt(1);
result.theta = x_opt(2);
result.t_drops = [x_opt(3), x_opt(5), x_opt(7)];
result.t_fuses = [x_opt(4), x_opt(6), x_opt(8)];
result.objective_value = -fval;
result.assigned_missile = assigned_missile;
uav_vel = [result.v_uav * cos(result.theta), result.v_uav * sin(result.theta), 0];
result.drop_points = zeros(3, 3);
result.burst_points = zeros(3, 3);
for i = 1:3

```

```

        drop_point = uav_initial + uav_vel * result.t_drops(i);
        result.drop_points(i, :) = drop_point;
        burst_x = drop_point(1) + uav_vel(1) * result.t_fuses(i);
        burst_y = drop_point(2) + uav_vel(2) * result.t_fuses(i);
        burst_z = drop_point(3) + uav_vel(3) * result.t_fuses(i) - 0.5 * g * result.t_fuses(i)^2;
        result.burst_points(i, :) = [burst_x, burst_y, burst_z];
    end
    missile_path = @(t) missile_initial + missile_dir * v_missile * t;
    result.missile_coverage = calculate_coverage_time_final(...
        result.burst_points, result.t_drops + result.t_fuses, missile_path, ...
        P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration);
end
function f = uav_objective_function_final(...
    x, uav_initial, missile_dir, missile_initial, T_fake_target, ...
    P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration, ...
    g, v_missile, min_drop_interval, missile_time)
    v_uav = x(1);
    theta = x(2);
    t_drops = [x(3), x(5), x(7)];
    t_fuses = [x(4), x(6), x(8)];
    if any(t_drops(2:3) - t_drops(1:2) < min_drop_interval) || ...
        any(t_drops + t_fuses > missile_time)
        f = 1e6; % 惩罚违反约束的解
        return;
    end
    uav_vel = [v_uav * cos(theta), v_uav * sin(theta), 0];
    drop_points = zeros(3, 3);
    burst_points = zeros(3, 3);
    burst_times = zeros(1, 3);

    for i = 1:3
        drop_points(i, :) = uav_initial + uav_vel * t_drops(i);
        burst_x = drop_points(i, 1) + uav_vel(1) * t_fuses(i);
        burst_y = drop_points(i, 2) + uav_vel(2) * t_fuses(i);

```

```

        burst_z = drop_points(i, 3) + uav_vel(3) * t_fuses(i) - 0.5 * g * t_fuses(i)^2;
        burst_points(i, :) = [burst_x, burst_y, burst_z];
        burst_times(i) = t_drops(i) + t_fuses(i);
    end
    missile_path = @(t) missile_initial + missile_dir * v_missile * t;
    coverage = calculate_coverage_time_final(...
        burst_points, burst_times, missile_path, ...
        P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration);
    penalty = 0;
    for i = 1:3
        burst_point = burst_points(i, :);
        missile_at_burst = missile_initial + missile_dir * v_missile * burst_times(i);
        missile_to_target = T_fake_target - missile_at_burst;
        missile_to_smoke = burst_point - missile_at_burst;
        projection = dot(missile_to_smoke, missile_to_target) / norm(missile_to_target);
        if projection < 0 || projection > norm(missile_to_target)
            penalty = penalty + 10000; % 大幅增加惩罚
        end
        missile_height_at_burst = missile_at_burst(3);
        if abs(burst_point(3) - missile_height_at_burst) > 500 % 高度差超过 500 米
            penalty = penalty + 5000;
        end
        missile_path_dir = missile_dir;
        smoke_to_missile_path = burst_point - missile_at_burst;
        perpendicular_dist = norm(smoke_to_missile_path - dot(smoke_to_missile_path,
missile_path_dir) * missile_path_dir);
        if perpendicular_dist > 1000 % 距离导弹飞行路径超过 1000 米
            penalty = penalty + 5000;
        end
    end
    f = -coverage + penalty;
end
function coverage = calculate_coverage_time_final(...
    burst_points, burst_times, missile_path, ...

```

```

P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration)
dt = 0.05;
coverage = 0;
t_max = 120;
sample_points = generate_dense_sample_points(P_target_center, r_target, h_target);

% 时间仿真
for t = 0:dt:t_max
    M_t = missile_path(t);
    smoke_positions = [];
    for i = 1:size(burst_points, 1)
        if t >= burst_times(i) && t <= burst_times(i) + smoke_duration
            time_after_burst = t - burst_times(i);
            smoke_pos = burst_points(i, :) - [0, 0, v_sink * time_after_burst];
            smoke_positions = [smoke_positions; smoke_pos];
        end
    end
    if isempty(smoke_positions)
        continue;
    end

    % 检查目标是否被完全遮蔽
    if is_target_covered_final(M_t, smoke_positions, R_smoke, sample_points)
        coverage = coverage + dt;
    end
end
end

function points = generate_dense_sample_points(center, radius, height)
    points = [];
    points = [points; center(1), center(2), 0];
    points = [points; center(1), center(2), height];
    angles = 0:pi/8:2*pi;
    for i = 1:length(angles)-1
        angle = angles(i);

```

```

        x = center(1) + radius * cos(angle);
        y = center(2) + radius * sin(angle);

        % 底部边缘点
        points = [points; x, y, 0];

        % 顶部边缘点
        points = [points; x, y, height];

        % 侧面中点（不同高度）
        for h = [height/4, height/2, 3*height/4]
            points = [points; x, y, h];
        end
    end
end

function covered = is_target_covered_final(M, smoke_centers, R_smoke, sample_points)
    covered = true;

    for i = 1:size(sample_points, 1)
        P = sample_points(i, :);
        ray_origin = M;
        ray_direction = P - M;
        ray_length = norm(ray_direction);
        ray_direction = ray_direction / ray_length;

        point_covered = false;

        for j = 1:size(smoke_centers, 1)
            C = smoke_centers(j, :);
            [intersect, t] = ray_sphere_intersection(ray_origin, ray_direction, C, R_smoke);

            if intersect && t > 0 && t < ray_length
                point_covered = true;
                break;
            end
        end
    end
end

```

```

        end
    end

    if ~point_covered
        covered = false;
        return;
    end
end
end

function [intersect, t] = ray_sphere_intersection(ray_origin, ray_direction, sphere_center,
sphere_radius)
    oc = ray_origin - sphere_center;
    a = dot(ray_direction, ray_direction);
    b = 2 * dot(oc, ray_direction);
    c = dot(oc, oc) - sphere_radius^2;
    discriminant = b^2 - 4*a*c;

    if discriminant < 0
        intersect = false;
        t = inf;
    else
        t1 = (-b - sqrt(discriminant)) / (2*a);
        t2 = (-b + sqrt(discriminant)) / (2*a);

        if t1 >= 0
            t = t1;
            intersect = true;
        elseif t2 >= 0
            t = t2;
            intersect = true;
        else
            intersect = false;
            t = inf;
        end
    end
end

```

```

end
end
function effectiveness = evaluate_simultaneous_coverage_final(...
    results, missiles, missile_dirs, missile_times, ...
    P_target_center, r_target, h_target, R_smoke, v_sink, smoke_duration, g, v_missile)
    dt = 0.05;
    % 确定仿真时间范围
    t_min = 0;
    t_max = max(missile_times) + smoke_duration;
    t_sim = t_min:dt:t_max;

    % 收集所有烟幕弹的起爆点和起爆时间
    all_burst_points = [];
    all_burst_times = [];

    for uav_id = 1:5
        result = results{uav_id};
        all_burst_points = [all_burst_points; result.burst_points];
        all_burst_times = [all_burst_times, result.t_drops + result.t_fuses];
    end
    sample_points = generate_dense_sample_points(P_target_center, r_target, h_target);
    simultaneous_coverage = 0;
    for t = t_sim
        % 计算三枚导弹的位置
        M1_t = missiles{1} + missile_dirs{1} * v_missile * t;
        M2_t = missiles{2} + missile_dirs{2} * v_missile * t;
        M3_t = missiles{3} + missile_dirs{3} * v_missile * t;

        smoke_positions = [];
        for i = 1:size(all_burst_points, 1)
            if t >= all_burst_times(i) && t <= all_burst_times(i) + smoke_duration
                time_after_burst = t - all_burst_times(i);
                smoke_pos = all_burst_points(i, :) - [0, 0, v_sink * time_after_burst];
                smoke_positions = [smoke_positions; smoke_pos];
            end
        end
    end
end

```

```

        end
    end

    % 如果没有烟幕弹，跳过
    if isempty(smoke_positions)
        continue;
    end

    % 检查三枚导弹是否同时被遮蔽
    M1_covered = is_target_covered_final(M1_t, smoke_positions, R_smoke,
sample_points);
    M2_covered = is_target_covered_final(M2_t, smoke_positions, R_smoke,
sample_points);
    M3_covered = is_target_covered_final(M3_t, smoke_positions, R_smoke,
sample_points);

    if M1_covered && M2_covered && M3_covered
        simultaneous_coverage = simultaneous_coverage + dt;
    end
end

effectiveness = simultaneous_coverage;
end
final_complete_multi_uav_smoke_strategy();

```